

Vorlesung

Theoretische Informatik

(04 Automaten mit ϵ -Transitionen, Minimierung von DFA)

Alois Heinz
Hochschule Heilbronn,
Max-Planck-Str. 39, 74081 Heilbronn
heinz@hs-heilbronn.de

31. März/10. April 2026

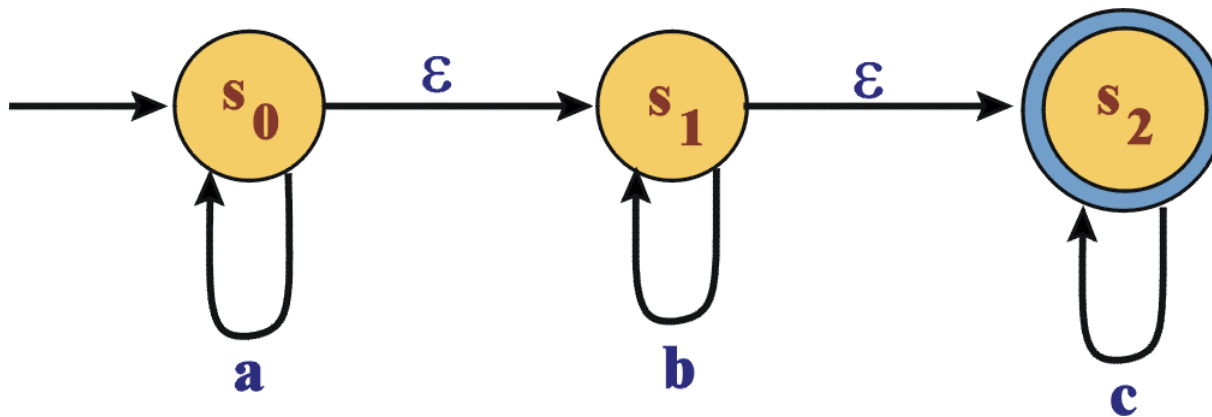
Motivation für ϵ -Übergänge

- Bei DFA und NFA wird bei jedem Zustandsübergang immer genau ein Zeichen aus Σ verbraucht.
- Für bestimmte Aufgaben lassen sich die Automaten einfacher entwerfen, wenn es möglich ist, **Nachfolgezustände durch ϵ -Transitionen** zu erreichen.

Bsp: Entwerfe Automaten $A_{\epsilon abc}$ für die Sprache

$$L_{abc} = \{a^i b^j c^k \mid i, j, k \geq 0\}$$

- $A_{\epsilon abc}$ darf beliebig viele a lesen, dann beliebig viele b , anschließend beliebig viele c
- Problem: Woher weiß $A_{\epsilon abc}$, wann eine Folge gleichartiger Zeichen zuende ist?
- Lösung: $A_{\epsilon abc}$ könnte irgendwann **ohne zu lesen** in neuen Zustand umschalten



Endlicher Automat mit ϵ -Übergängen

Ein **Endlicher ϵ -Automat** (ϵ FA) ist ein Quintupel

$$A = (\Sigma, S, \delta, S_0, F)$$

Dabei sind Σ , S , S_0 und F wie bei NFA definiert, und δ ist die **Zustandsüberführungsrelation**, die ϵ -Transitionen zulässt:

$$\delta \subseteq S \times (\Sigma \cup \{\epsilon\}) \times S$$

- Konfigurationen $k = (s, w)$ sind wie bei NFA definiert
- Für Konfigurationsübergänge gilt:

$$(s, aw) \vdash (t, w) \text{ g.d.w. } (s, a, t) \in \delta, a \in \Sigma \cup \{\epsilon\}, w \in \Sigma^*$$

- Die von einem ϵ FA A akzeptierte Sprache ist dann wieder

$$L(A) = \{w \in \Sigma^* \mid (s_0, w) \vdash^* (s, \epsilon), s_0 \in S_0, s \in F\}$$

- Die Klasse der Sprachen über Σ , die von ϵ FA akzeptiert werden, ist ϵFA_Σ

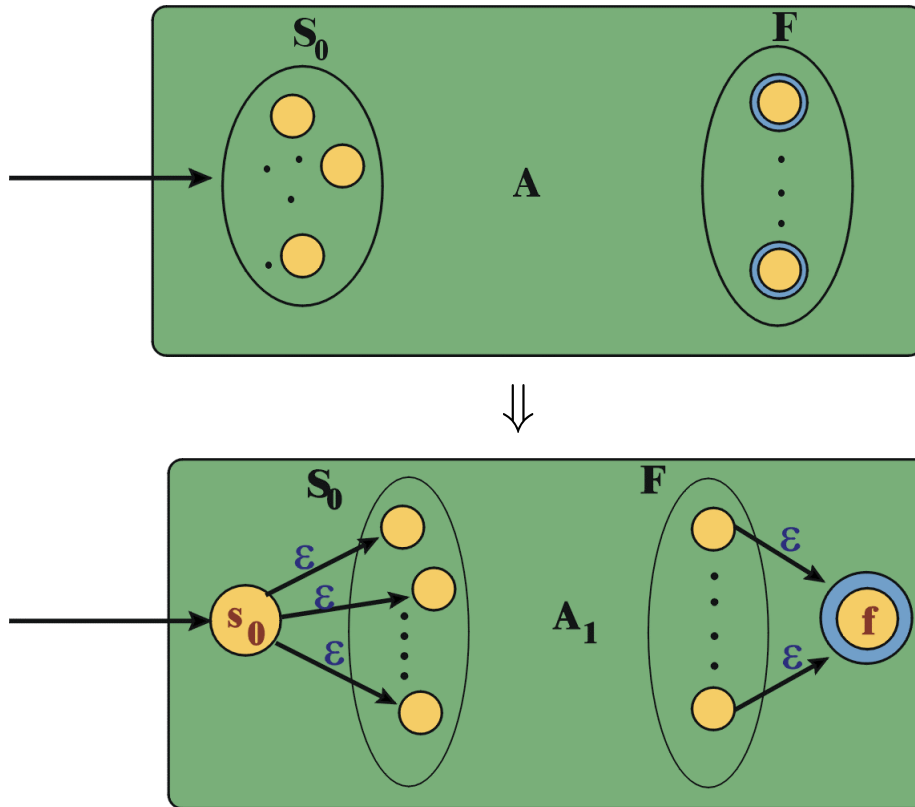
Äquivalenz von ϵ -Automaten und NFA

Satz. Für beliebiges Alphabet gilt: $\epsilon FA_{\Sigma} = NFA_{\Sigma}$

- $NFA_{\Sigma} \subseteq \epsilon FA_{\Sigma}$, da jeder NFA ein ϵ FA (ohne Epsilon-Übergänge) ist.
- Zu zeigen bleibt: $\epsilon FA_{\Sigma} \subseteq NFA_{\Sigma}$
- Dazu wird ein ϵ FA $A = (\Sigma, S, \delta, S_0, F)$ in vier Schritten in einen äquivalenten NFA transformiert.

Transformation von ϵ FA in NFA, 1. Schritt

A erhält **einen** neuen Startzustand s_0 und **einen** neuen Endzustand f , welche mit den bisherigen Start- bzw. Endzuständen über ϵ -Transitionen verbunden werden:

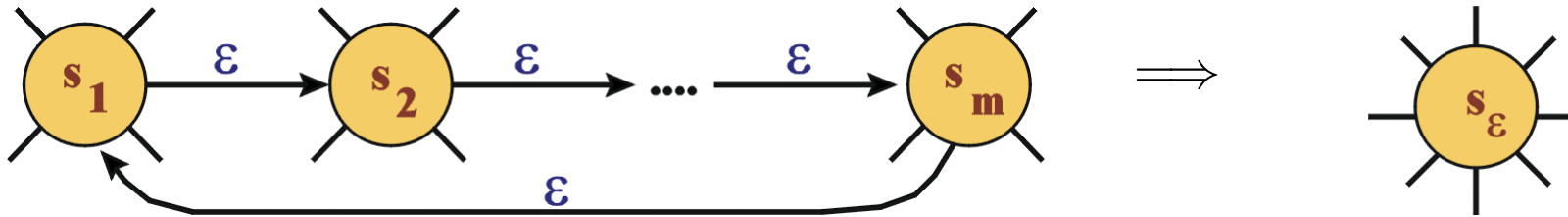


(Falls ohnehin nur ein Startzustand oder nur ein Endzustand vorhanden ist, kann auf das Erneuern verzichtet werden.)

Transformation von ϵ FA in NFA, 2. Schritt

Entferne alle ϵ -Zykel, Zustandsfolgen $s_1, s_2, \dots, s_m$, die zyklisch über ϵ -Transitionen durchlaufen werden können, ersetze sie jeweils durch neuen Zustand s_ϵ .

Zu und von s_ϵ führen alle Transitionen der ersetzten Zustände.

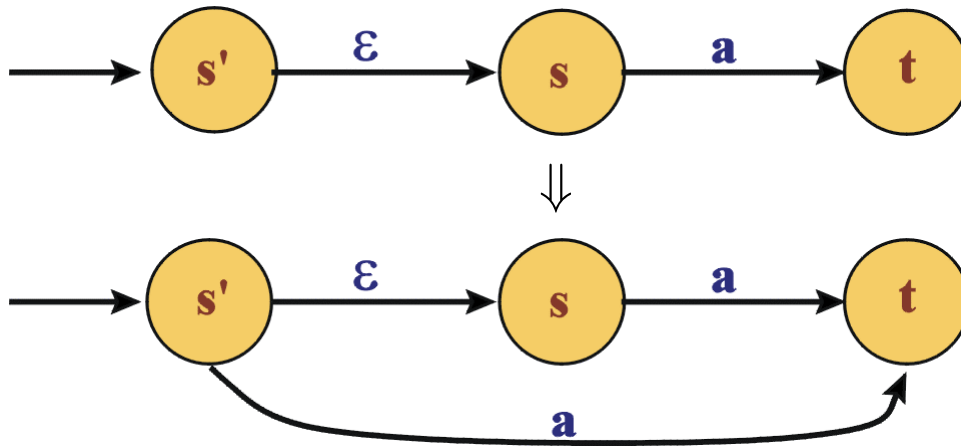


Anschließend werden alle Transitionen der Art (s, ϵ, s) entfernt.

Transformation von ϵ FA in NFA, 3. Schritt

Jetzt noch enthaltene ϵ -Transitionen werden durch **alternative Übergänge** ergänzt, die Zeichen aus Σ benötigen. (Ziel ist es, die ϵ -Transitionen überflüssig zu machen.)

Kommt man mit **<erst ϵ , dann a >** von s' nach t , dann auch mit **<nur a >**.



Die Operation wird so oft angewandt, bis sich keine Änderungen mehr ergeben.

Transformation von ϵ FA in NFA, 4. Schritt

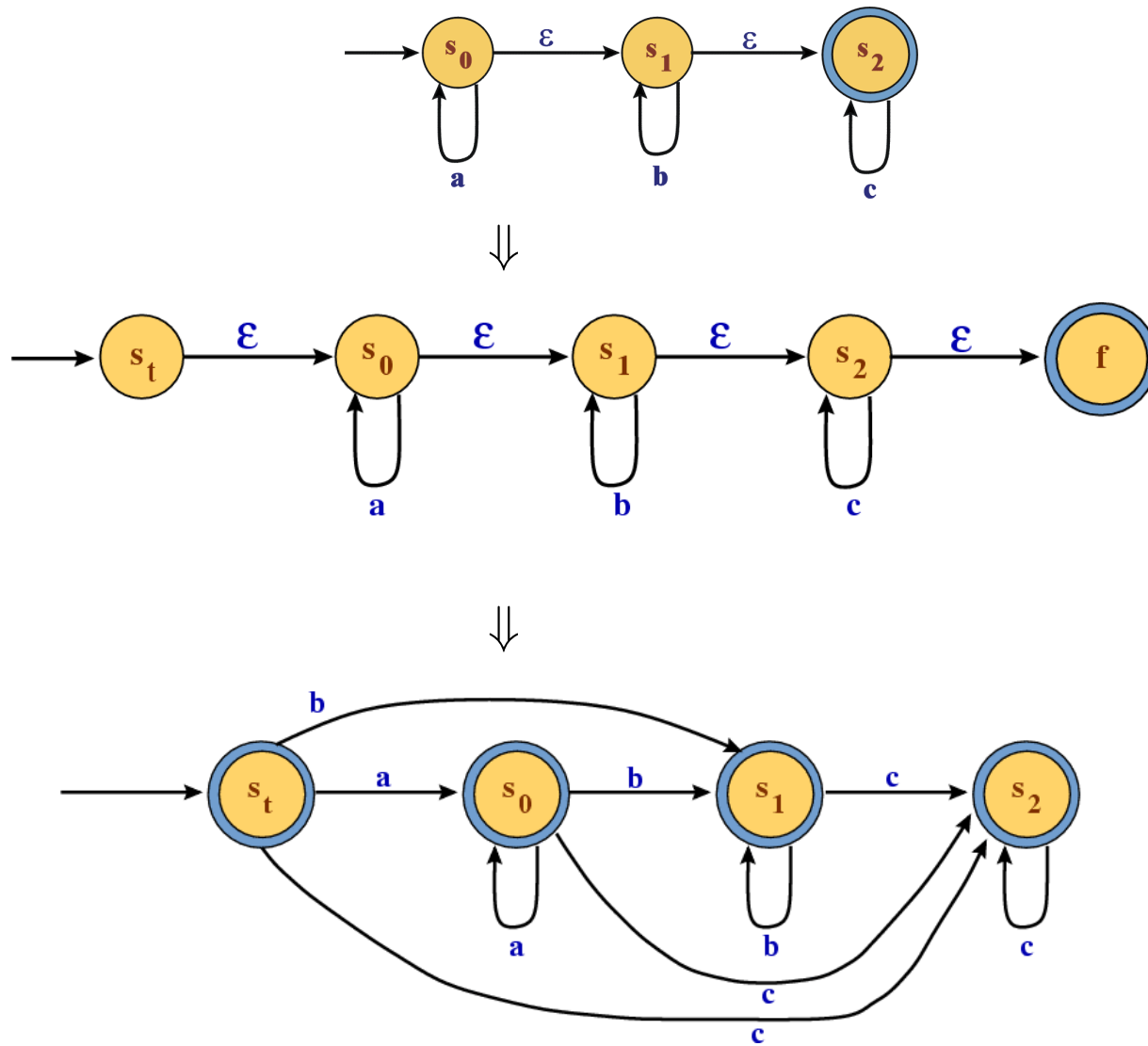
Ein neu eingefügter Endzustand f (der nur das Ziel von ϵ -Transitionen ist) wird wieder entfernt und in die **Endzustandmenge** F' die Zustände aufgenommen, von denen f per ϵ -Transitionen erreichbar war:

$$F' = \{t \in S \mid (t, \epsilon) \vdash^* (f, \epsilon)\}$$

Anschließend können **alle ϵ -Transitionen entfernt** werden.

Da alle Transformationsschritte äquivalenzerhaltend waren, besitzt der entstandene Automat die gleiche Sprache wie der Ausgangsautomat.

Bsp: Transformation von $A_{\epsilon abc}$ in einen NFA

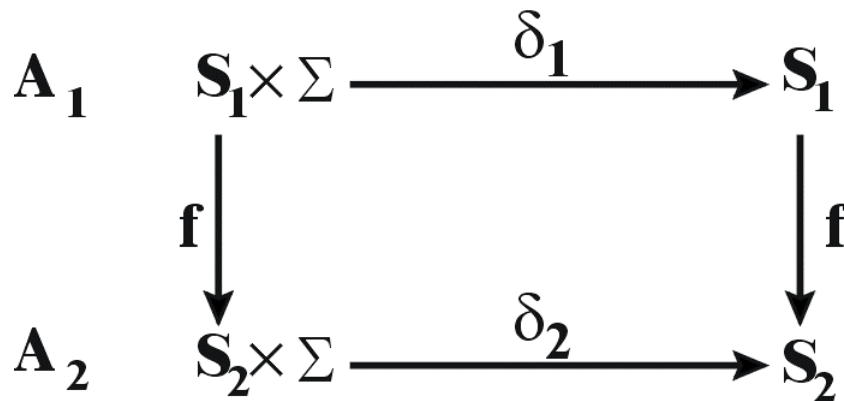


Minimaler endlicher Automat

Satz. Gegeben sei eine reguläre Sprache L über Σ , also $L \in REG_{\Sigma}$.
Dann ist der (Zustands-) minimale Automat $A_{\min}$, für den $L = L(A_{\min})$ gilt,
bis auf Isomorphie eindeutig bestimmt.

Dabei sind zwei Automaten $A_1 = (\Sigma, S_1, \delta_1, s_{0_1}, F_1)$ und $A_2 = (\Sigma, S_2, \delta_2, s_{0_2}, F_2)$ genau dann isomorph ($A_1 \cong A_2$), wenn $|S_1| = |S_2|$ und wenn es eine bijektive Abbildung $f : S_1 \rightarrow S_2$ gibt mit

$$\begin{aligned} f(\delta_1(s, a)) &= \delta_2(f(s), a) \quad \forall s \in S_1, \forall a \in \Sigma \\ f(F_1) &= F_2 \end{aligned}$$



(Kommutatives Diagramm)

Minimierung endlicher Automaten

Sei $A = (\Sigma, S, \delta, s_0, F)$ irgend ein DFA, der $L = L(A)$ erkennt.

Zu A wird in mehreren Schritten ein äquivalenter Automat $A_{\min}$ konstruiert:

1. Vereinfache A , so dass alle Zustände **erreichbar** sind.
2. **Zerlege** die Zustandsmenge disjunkt in 2 Teile: $\Pi_1 = \{F, S - F\}$
3. **Verfeinere** die aktuelle Zerlegung $\Pi_i = \{S_1, \dots, S_k\}$: In der neuen Zerlegung Π_{i+1} gehören Zustände s, s' genau dann zur gleichen Menge, wenn $s \in S_i$ und $s' \in S_i$ sowie $\delta(s, a) \in S_j$ und $\delta(s', a) \in S_j$ für alle $a \in \Sigma$ und $i, j \in \{1, \dots, k\}$.
Aufgeteilt werden muss S_i , wenn für $s, s' \in S_i$ gilt $\delta(s, a) \in S_j \neq S_{j'} \ni \delta(s', a)$ für ein $a \in \Sigma$.
4. Ergab die letzte Verfeinerung mehr Mengen, gehe zurück zu 3.
Andernfalls: Die Mengen der letzten Zerlegung sind die Zustände von $A_{\min}$.
 $s_{0_{\min}}$, $F_{\min}$ und $\delta_{\min}$ werden auf naheliegende Weise definiert.

Beispiel: Minimierung endlicher Automaten

		M_1		M_2		
Π_1		s_3	s_4	s_0	s_1	s_2
	$\delta(0)$	M_2	M_2	M_2	M_2	M_2
	$\delta(1)$	M_1	M_1	M_2	M_1	M_1
		M_1		M_2	M_3	
Π_2		s_3	s_4	s_0	s_1	s_2
	$\delta(0)$	M_3	M_3	M_3	M_3	M_3
	$\delta(1)$	M_1	M_1	M_3	M_1	M_1
Π_3	$= \Pi_2$					

